

TRABAJO FINAL INTEGRADOR

Primera Entrega - Propuesta de Proyecto y Repositorio

Sistema SaaS para Gestión de Servicios Técnicos

Plataforma multi-organización para trazabilidad integral de reparaciones

Carrera	Tecnicatura Universitaria en Programación a Distancia
Integrantes	Matías Ariel Deluca · Gastón Armando Giorgio · Andrés Emanuel Meshler
Tutora	María Candela Grosso
Fecha de entrega	30/08/2026
Repositorio GitHub	Repositorio Github

1. Resumen ejecutivo

La propuesta consiste en desarrollar una plataforma SaaS para la gestión de servicios técnicos, orientada inicialmente a pequeños y medianos negocios que reciben equipos para diagnóstico, presupuesto, reparación y entrega. **El objetivo es centralizar información hoy dispersa entre papel, planillas, mensajería y registros aislados, mejorando la trazabilidad de cada reparación y el control interno del negocio.**

La plataforma será multi-organización: distintos servicios técnicos utilizarán la misma aplicación, pero los datos de cada negocio permanecerán lógicamente aislados. El MVP priorizará autenticación, usuarios y roles, clientes, equipos, órdenes de reparación, asignación de técnicos, diagnóstico, presupuesto, estados, historial y entrega.

Propuesta de valor en una frase

Unificar en una sola plataforma el ciclo completo de una reparación, desde la recepción del equipo hasta su entrega, con trazabilidad por estado, responsables y organización.

2. Problemática identificada

En muchos servicios técnicos de pequeña y mediana escala, la información operativa se administra mediante herramientas no integradas: conversaciones de WhatsApp, anotaciones en papel, planillas, archivos locales o sistemas genéricos. El problema no es la ausencia de una aplicación en sí misma, sino la falta de una fuente única y trazable para seguir cada trabajo.

- La información de clientes, equipos y reparaciones puede quedar fragmentada en distintos medios.

- El estado actual de una reparación puede depender de consultas manuales o del conocimiento de una persona específica.
- Los cambios de responsable, diagnóstico, presupuesto o estado pueden no quedar registrados cronológicamente.
- La asignación de técnicos y el seguimiento de trabajos abiertos puede resultar difícil de visualizar.
- La recuperación del historial de un cliente o equipo puede demandar búsqueda manual y generar duplicaciones.

Impacto esperado: mayor esfuerzo administrativo, riesgo de pérdida o duplicación de información, menor visibilidad del estado de los trabajos y demoras para responder consultas internas o de clientes. En esta etapa no se inventan métricas cuantitativas: el relevamiento con usuarios del dominio se utilizará para validar y medir estos impactos.

2.1. Flujo actual simplificado y puntos de dolor

Etapa	Práctica frecuente	Punto de dolor
Recepción	Registro manual o mensaje	Datos incompletos o difíciles de recuperar
Diagnóstico	Comunicación interna informal	Falta de responsable/fecha claramente registrada
Presupuesto	Mensaje o archivo separado	Dificultad para vincularlo con la orden y su estado
Reparación	Seguimiento por memoria o chat	Baja visibilidad de avances y pendientes
Entrega	Cierre manual	Historial disperso y poca trazabilidad posterior

3. Actores y necesidades

Actor	Responsabilidad principal	Necesidades en el MVP
Propietario / Administrador	Configura el servicio técnico y supervisa la operación	Gestionar usuarios/roles, visualizar órdenes, controlar actividad y dashboard
Recepcionista	Recibe equipos y registra la información inicial	Gestionar clientes/equipos, crear órdenes, asignar técnicos, consultar estados
Técnico	Diagnostica y repara equipos asignados	Ver órdenes asignadas, registrar diagnóstico, observaciones y avances
Cliente	Actor beneficiario, sin acceso directo en el MVP	Recibir una atención más ordenada y un seguimiento consistente

3.1. Validación inicial del problema

La propuesta parte de una ineficiencia observable y común en negocios pequeños: la coexistencia de herramientas manuales o desconectadas. Para evitar presentar como hechos datos no relevados, el equipo realizará una validación temprana mediante entrevistas breves a personas vinculadas con servicios técnicos, enfocadas en el flujo actual, información perdida, tareas repetitivas y tiempo dedicado al seguimiento.

4. Solución propuesta

Se desarrollará una aplicación web SaaS que permita a cada servicio técnico crear su organización y operar dentro de un espacio propio. Los usuarios autenticados accederán a funciones según su rol y todas las operaciones del negocio quedarán asociadas a su organización.

Modelo SaaS multi-organización



Figura 1. Separación lógica de datos entre organizaciones dentro de una misma plataforma.

4.1. Diferenciación

Alternativa	Centralización	Trazabilidad de estados	Roles	Multiorganización	Adecuación al flujo técnico
Papel + mensajería	Baja	Baja	No	No	Parcial
Planilla de cálculo	Media	Baja / manual	No	No	Parcial
Sistema genérico de gestión	Alta	Variable	Variable	Variable	No especializada
Propuesta SaaS	Alta	Alta	Sí	Sí	Sí

5. Objetivos

5.1. Objetivo general

Diseñar, implementar y desplegar un MVP web para la gestión integral de servicios técnicos que centralice clientes, equipos y órdenes de reparación, permita gestionar responsabilidades y estados, y conserve un historial trazable de cada trabajo dentro de un esquema SaaS multi-organización.

5.2. Objetivos específicos

- Implementar autenticación segura y autorización por roles para los usuarios de cada organización.

- Permitir registrar y administrar clientes y equipos asociados a cada servicio técnico.
- Gestionar el ciclo de vida de una orden de reparación desde la recepción hasta la entrega.
- Permitir asignar técnicos y registrar diagnósticos, presupuestos, observaciones y cambios de estado.
- Mantener un historial cronológico de cambios relevantes para mejorar la trazabilidad.
- Garantizar el aislamiento lógico de la información entre organizaciones dentro de la plataforma.
- Desplegar al menos los componentes principales en servicios online y documentar instalación, uso y arquitectura.

6. Alcance del MVP

Módulo	Incluido en el MVP
Autenticación	Registro/inicio de sesión, emisión y validación de JWT, protección de rutas
Organizaciones	Creación del servicio técnico y asociación de usuarios a una organización
Usuarios y roles	Administrador, recepcionista y técnico con permisos diferenciados
Clientes	Alta, consulta, edición y baja lógica/gestión según decisión de diseño
Equipos	Registro de tipo, marca, modelo, número de serie y vínculo con cliente
Órdenes	Creación, consulta, edición controlada, asignación de técnico y responsable
Diagnóstico	Registro del problema detectado, solución propuesta y observaciones
Presupuesto	Importes/conceptos principales y estado
	pendiente/aprobado/rechazado
Estados e historial	Seguimiento cronológico del ciclo de reparación
Entrega	Registro de cierre y entrega del equipo
Dashboard básico	Indicadores operativos simples por estado y volumen de órdenes

6.1. Flujo de negocio principal

Flujo principal de una orden de reparación



Figura 2. Flujo propuesto del ciclo de una orden de reparación.

6.2. Fuera de alcance de la primera versión

- Pagos online y suscripciones comerciales del SaaS.
- Aplicación móvil nativa.
- Integración oficial con WhatsApp, SMS u otros proveedores externos.
- Portal de autoservicio para clientes finales.
- Facturación electrónica o integración con controladores fiscales.
- Gestión avanzada de inventario de repuestos y compras.
- Reportes analíticos avanzados o funcionalidades de inteligencia artificial.

7. Stack tecnológico y justificación

Capa	Tecnología	Justificación
Frontend	React + TypeScript	Aplicación web modular y mantenible. TypeScript aporta tipado estático y mejora la detección temprana de errores.
Backend	Java + Spring Boot	Framework maduro para APIs web con lógica de negocio, estructura por capas y fuerte soporte de seguridad y validación.
Persistencia	Spring Data MongoDB	Integra el modelo documental de MongoDB con repositorios y servicios de Spring, evitando usar JPA/Hibernate sobre una base NoSQL.
Base de datos	MongoDB Atlas	Base documental administrada en la nube, adecuada para documentos con información jerárquica y evolución progresiva del modelo.
Seguridad	Spring Security + JWT	Permite autenticación y autorización de una API REST, aplicando permisos por rol y protección de endpoints.

Despliegue	Vercel + Render + MongoDB Atlas	Separación simple de frontend, backend y base de datos con componentes accesibles online.
Versionado	GitHub	Repositorio único exigido por la asignatura; centraliza código, documentación y avances.

7.1. Justificación de MongoDB

MongoDB se utilizará como base NoSQL documental. La decisión se apoya en que una orden de reparación puede concentrar información jerárquica como diagnóstico, presupuesto, observaciones e historial de estados. Esto permite modelar parte de la información relacionada dentro del mismo agregado documental cuando resulte conveniente, mientras que clientes, equipos, usuarios y organizaciones podrán mantenerse referenciados. El diseño definitivo de documentos, referencias, índices y validaciones se presentará en la segunda entrega.

7.2. Arquitectura preliminar

Arquitectura propuesta del MVP

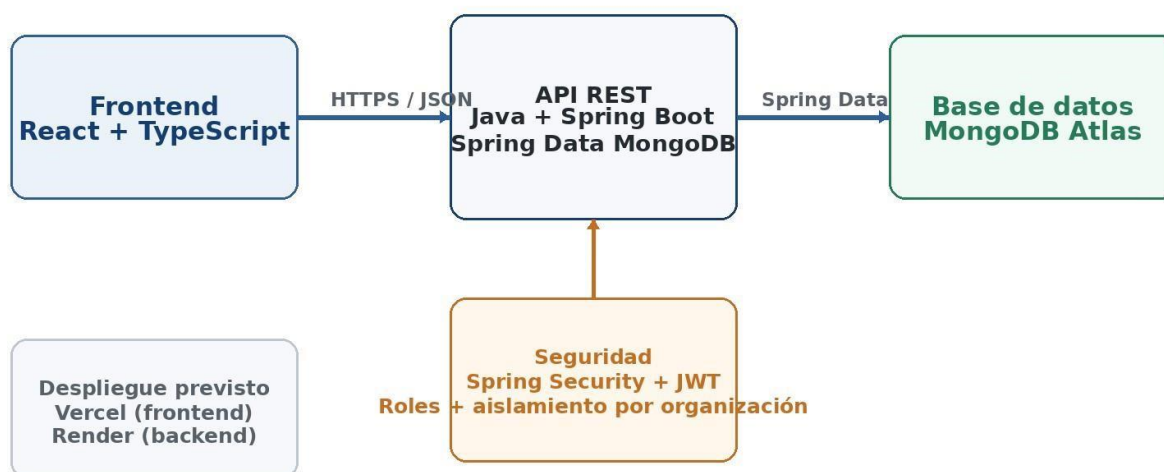


Figura 3. Arquitectura preliminar de la solución.

7.3. Aislamiento multi-organización

Cada documento de negocio que requiera aislamiento incluirá un identificador de organización (por ejemplo, `organizationId`). La API utilizará el contexto del usuario autenticado para restringir consultas y operaciones a su organización. Esta regla se aplicará en la capa de servicio/repositorio y será tratada como requisito transversal de seguridad, evitando que un usuario pueda consultar recursos pertenecientes a otro servicio técnico.

8. Análisis de viabilidad

Dimensión	Evaluación	Fundamento
-----------	------------	------------

Técnica	Viable	El stack está compuesto por tecnologías maduras. Spring Boot y React son apropiados para una aplicación web con lógica de negocio; Spring Data MongoDB integra el backend con Atlas.
Operativa	Viable con validación temprana	La solución se adapta a un flujo reconocible de recepción-diagnóstico-presupuesto-reparación-entrega. Se prevén entrevistas breves para validar necesidades y terminología.
Temporal	Viable si se respeta el MVP	El alcance excluye integraciones externas, pagos, app móvil y reportes complejos. El desarrollo se divide por módulos y prioriza el flujo principal.
Económica	Viable para el prototipo académico	Se prevé utilizar servicios cloud con planes adecuados para desarrollo/prototipado, evitando infraestructura propia en la primera versión.

8.1. Riesgos iniciales y mitigaciones

Riesgo	Impacto	Mitigación
Curva de aprendizaje en Spring Security/JWT	Medio	Implementar autenticación en una etapa temprana y validar con pruebas de integración.
Diseño incorrecto del modelo documental	Alto	Prototipar documentos y consultas; revisar embebidos vs referencias antes de la segunda entrega.
Fallas de aislamiento entre organizaciones	Alto	Definir organizationId como regla transversal, pruebas de autorización y casos negativos.
Sobrecarga de funcionalidades	Alto	Mantener el no-alcance explícito y priorizar el flujo principal de órdenes.
Retrasos por integración frontend/backend	Medio	Definir contrato de API/DTOs temprano y trabajar con endpoints incrementales.
Problemas de despliegue al final	Medio	Realizar un despliegue mínimo temprano y actualizarlo durante el proyecto.

9. Criterios de éxito del MVP

- Un servicio técnico puede registrarse/iniciar sesión y operar sin acceder a datos de otra organización.
- Los roles definidos restringen correctamente las acciones sensibles de acuerdo con sus permisos.
- Un usuario autorizado puede registrar un cliente, asociar un equipo y crear una orden de reparación.
- La orden puede recorrer el flujo principal registrando técnico, diagnóstico, presupuesto, estados e historial.
- La información persiste en MongoDB Atlas y puede recuperarse correctamente desde la aplicación web.
- Frontend y backend se encuentran desplegados y accesibles mediante servicios online.

- El repositorio contiene instrucciones de instalación, configuración, arquitectura, documentación de API y entregas académicas.

10. Plan de trabajo

Período	Objetivo / entregable principal
Hasta 30/08	Primera entrega: problemática, propuesta, alcance, stack, viabilidad, plan y repositorio único.
31/08 - 07/09	Refinamiento de requerimientos, backlog, arquitectura base, estructura frontend/backend y primer despliegue de prueba.
08/09 - 20/09	Autenticación, organizaciones, usuarios/roles y diseño del modelo documental. Pruebas iniciales.
21/09 - 27/09	Cierre de segunda entrega: esquema de base de datos, índices preliminares y listado de módulos aprobados.
28/09 - 11/10	Cientes, equipos y órdenes de reparación. Integración frontend/backend.
12/10 - 25/10	Asignación de técnicos, diagnóstico, presupuesto, estados e historial.
26/10 - 01/11	Entrega/cierre de órdenes, dashboard básico y endurecimiento
	de permisos multi-organización.
02/11 - 08/11	Pruebas funcionales, correcciones, despliegue estable y documentación técnica.
09/11 - 14/11	Informe final, README definitivo, demo, video explicativo y revisión de entrega.

10.1. Metodología de trabajo

El equipo trabajará de forma incremental, con un backlog priorizado y seguimiento mediante GitHub Projects o tablero Kanban. Las funcionalidades se desarrollarán por ramas de trabajo y se integrarán mediante revisiones antes de incorporarse a la rama principal. Las decisiones relevantes de arquitectura, alcance y base de datos quedarán documentadas en el repositorio.

11. Repositorio único de GitHub

Todo el proyecto se centralizará en un único repositorio, conforme al requisito de la asignatura. Estructura inicial propuesta:

/

```

├── frontend/           # React + TypeScript
├── backend/           # Java + Spring Boot
├── docs/
│   ├── entregas/      # PDF y documentación de cada hito
│   ├── arquitectura/  # Diagramas y decisiones técnicas
│   └── base-datos/    # Modelo documental, índices y ejemplos
└── README.md          # Descripción, instalación, stack e integrantes

```

```

└─ .gitignore
└─ LICENSE (si corresponde)

```

URL del repositorio
<https://github.com/ameshler/TFI-SaaS-Servicios-Tecnicos.git>

12. Uso de inteligencia artificial

La IA se utilizará como herramienta de apoyo para refinamiento de requerimientos, detección de riesgos, generación de casos de prueba, revisión de código, documentación y refactorización. Las decisiones de arquitectura, seguridad, modelo de datos y lógica de negocio serán revisadas y justificadas por el equipo, manteniendo trazabilidad de los cambios relevantes.

13. Base metodológica utilizada

La propuesta fue estructurada siguiendo el material de la Unidad 1 de la asignatura:

- U1-A1 - Identificación de una problemática y propuesta de solución: contexto del problema, actores, valor agregado y validación.
- U1-A2 - Definición del stack tecnológico: elección justificada de frontend, backend, base de datos y despliegue.
- U1-A3 - Propuesta y Viabilidad asistida por IA: definición de MVP, alcance/no alcance, riesgos, criterios de éxito y viabilidad técnica, operativa y temporal.

14. Checklist de cumplimiento de la primera entrega

Ítem esperado	Dónde se encuentra	Estado
Problemática y contexto	Secciones 2 y 2.1	Cumplido
Actores y necesidades	Sección 3	Cumplido
Propuesta de solución y valor agregado	Secciones 1 y 4	Cumplido
Objetivo general y específicos	Sección 5	Cumplido
Alcance / MVP y no alcance	Sección 6	Cumplido
Stack tecnológico y justificación	Sección 7	Cumplido
Plataforma / nube	Secciones 7 y 8	Cumplido
Viabilidad y riesgos	Sección 8	Cumplido
Criterios de éxito	Sección 9	Cumplido
Plan de trabajo	Sección 10	Cumplido
Repositorio único de GitHub	Sección 11	Cumplido

Conclusión

El proyecto presenta un problema concreto, un flujo de negocio comprensible, un MVP acotado y un stack técnicamente coherente. La principal condición para mantener la viabilidad temporal es evitar incorporar funcionalidades fuera del alcance antes de completar el ciclo básico de reparación, la seguridad por roles y el aislamiento entre organizaciones.